PREPROCESSING DATASET KLASIFIKASI UANG KERTAS RUPIAH

Dosen Pengampu:

Erika Maulidiya, S.Kom, M.Kom

Disusun Oleh:

Dina Izzati Elfadheya NIM 2310817120001

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

2026

1. LATAR BELAKANG DAN URGENSI PEMILIHAN DATASET

Hidup di era digital dengan sistem pembayaran cashless yang semakin popular ini membuat banyak orang lebih memilih pembayaran tersebut karena mudah dilakukan. Namun, kenyataannya perputaran uang tunai di masyarakat Indonesia masih sangat massif digunakan[1][2]. Dari kondisi inilah, ditemukan beberapa masalah nyata di lapangan yang menjadi alasan mengapa pembuatan dataset klasifikasi uang kertas ini penting untuk dilakukan. Masalah pertama yang paling sering terjadi adalah kesalahan manusia (*human error*) akibat kemiripan warna pada uang kertas[3]. Kasus yang kerap ditemui adalah tertukarnya pecahan Rp2.000 dengan Rp50.000, atau Rp75.000 dengan Rp100.000. Kesalahan ini rawan terjadi saat transaksi dilakukan dengan terburu-buru atau di tempat yang minim cahaya, seperti warung malam atau pasar tradisional[3]. Oleh karena itu, pembuatan dataset ini secara khusus menyertakan gambar augmentasi dengan kondisi cahaya gelap agar sistem kecerdasan buatan atau AI nanti tidak ikut terkecoh oleh minimnya pencahayaan seperti mata manusia.

Urgensi lain dari dataset ini adalah untuk membantu aksesibilitas penyandang tunanetra. Bank Indonesia sebenarnya telah memberikan garis timbul (*blind code*) pada setiap uang kertas agar nominalnya bisa diraba. Namun pada praktiknya, uang yang beredar dari tangan ke tangan sering kali sudah dalam kondisi lecek, kucel, atau aus sehingga garis timbul tersebut rata dan tidak bisa diraba lagi. Dengan adanya dataset yang mencakup berbagai sudut pandang dan kondisi uang ini, harapannya nanti dapat tercipta sebuag aplikasi pendeteksi uang berbasis kamera smartphone yang mampu bersuara untuk memfasilitasi kemandirian penyandang tunanetra dalam bertransaksi. Dari dataset ini diharapkan dapat menjadi solusi teknologi yang murah bagi pedagang kecil. Saat ini, mesin pendeteksi uang otomatis biasanya dibanderol dengan harga mahal dan hanya dimiliki oleh bank, mesin ATM, atau mesin kasir di minimarket besar[4]. Pelaku UMKM, pedagang kelontong, atau tukang parkir tidak memiliki akses ke alat tersebut. Dataset gambar ini merupakan fondasi awal yang krusial untuk membangun sistem deteksi uang otomatis yang ringan. Sehingga nantinya algoritma dari sistem klasifikasi ini kelak bisa diimplementasikan menjadi aplikasi yang terjangkau dan mudah diakses oleh masyarakat kecil untuk mengecek nominal uang pelanggan dengan cepat dan akurat.

2. DESKRIPSI DATASET

Dataset yang digunakan dalam penelitian ini merupakan sekumpulan gambar uang kertas Rupiah yang saya kumpulkan secara mandiri. Sebelum memasuki tahapan pemrosesan data (seperti kompresi ukuran atau modifikasi cahaya), dataset mentah ini memiliki karakteristik dan gambaran fisik sebagai berikut:

a. Definisi dan Kategori Kelas

Dataset ini merupakan kumpulan foto asli uang kertas Rupiah yang baru yang saat ini beredar luas di Masyarakat Indonesia. Objek uang di dalam dataset ini dibagi ke dalam delapan kategori kelas nominal yang berbeda, yaitu pecahan Rp1.000, Rp2.000, Rp5.000, Rp10.000, Rp20.000, Rp50.000, Rp75.000, dan Rp100.000.

b. Jumlah dan Distribusi Data Mentah

Sebelum dilakukan tahapan penggandaan data, dataset awal ini terdiri dari 192 lembar foto. Untuk mencegah bias saat sistem komputer belajar nantinya, saya merancang agar distribusi data awal ini sudah dalam kondisi seimbang (*balance*) sejak pertama kali dikumpulkan. Oleh karena itu, masing-masing dari delapan kelas pecahan uang tersebut diwakili dengan jumlah yang sama rata, yakni tepat 24 lembar foto yang unik per kelasnya.

c. Variasi Sudut Pandang (*Angle*)

Agar sistem nanti bisa mengenali bentuk fisik uang secara utuh, saya memoto uang tersebut dari berbagai arah, bukan hanya tegak lurus. Saya mengambil gambar dengan enam variasi sudut pandang yang berbeda untuk setiap pecahannya, yaitu dari arah depan, arah belakang, kemiringan sisi kiri, kemiringan sisi kanan, serta pengambilan gambar dari jarak dekat dan jarak yang sedikit menjauh.

d. Sisi Uang dan Variasi Latar Belakang

Setiap objek uang kertas difoto dengan menampilkan dua belah sisi yaitu sisi bagian depan yang biasanya menampilkan tokoh pahlawan nasional, serta sisi bagian belakang yang menampilkan gambar kebudayaan atau pemandangan alam. Selain itu, untuk membiasakan komputer mengenali batas fisik uang, saya meletakkan objek uang tersebut di atas dua jenis permukaan, yaitu

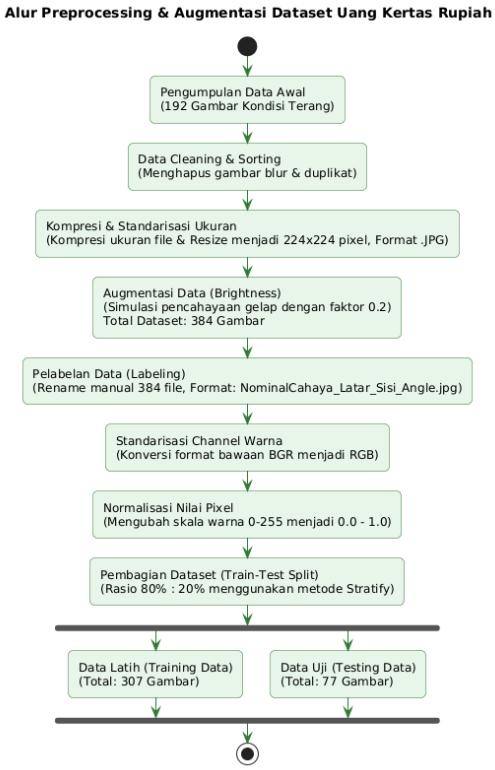
permukaan meja dengan latar belakang polos dan latar belakang yang memiliki motif batik menggunakan kain / tapih batik.

e. Perangkat dan Kondisi Pencahayaan Asli

Seluruh gambar awal ini difoto menggunakan kamera handphone A77s. yang supaya resolusi dan karakteristik gambarnya menyerupai foto yang umum diambil oleh masyarakat sehari-hari. Pada wujud aslinya ini, seluruh 192 gambar tersebut difoto dalam satu kondisi pencahayaan yang sama, yaitu difoto di dalam ruangan dengan kondisi cahaya yang terang.

3. TAHAPAN PREPROCESSING, AUGMENTASI, DAN PEMBAGIAN DATA

Berdasarkan diagram alur sistem yang telah dirancang, seluruh tahapan pemrosesan data dilakukan secara berurutan untuk menjaga efisiensi memori dan kualitas data.



1. Pengumpulan Data Awal dan Pembersihan (*Data Cleaning*)

Tahap pertama dalam penyusunan dataset ini dilakukan dengan rincian langkah sebagai berikut:

a. Pemilihan Alat Pengambilan Gambar

Pengambilan gambar dilakukan secara langsung menggunakan kamera kamera Handphone Oppo A177s. Pemilihan ini dilakukan agar hasil fotonya dapat mewakili kondisi asli dan natural dari transaksi sehari-hari di masyarakat

b. Pengambilan foto

Saya mengumpulkan total 192 gambar uang kertas yang semuanya diambil dalam kondisi pencahayaan ruangan yang terang. Angka 192 didapat dari 24 variasi gambar untuk masing-masing dari 8 nominal uang kertas (mulai dari pecahan Rp1.000 hingga Rp100.000).

c. Quality Control

Setelah ratusan foto tersebut terkumpul di galeri, proses selanjutnya adalah dengan tahapan pengawasan mutu (quality control) dengan mengelompokkan secara manual, atau biasa disebut data cleaning. Saya memeriksa foto-foto tersebut satu per satu untuk menyaring dan membuang gambar yang kualitasnya tidak memenuhi standar kelayakan untuk dipelajari oleh komputer.

d. Penerapan Kriteria Data Cleaning

Dalam membuang gambar yang kualitasnya tidak memenuhi standar, saya menerapkan empat kriteria yaitu:

- Buram (*Blur*): Foto yang hasilnya goyang atau tidak fokus akibat pergerakan kamera langsung dibuang.
- Silau atau *Overexposure*: Foto yang terlalu silau akibat pantulan cahaya lampu atau *flash* hingga menutupi tulisan angka nominal uang juga dihapus.
- Objek Terpotong: Saya memastikan seluruh bentuk fisik uang masuk ke dalam *frame* kamera. Jika ada bagian tepi uang yang terpotong di pinggir foto, maka gambar tersebut tidak dipakai.
- Gambar Duplikat: Saya menyeleksi dengan saksama agar tidak ada gambar yang posisinya nyaris sama persis (duplikasi) yang ikut tersimpan.

e. Pengambilan Foto Ulang

Jika ada foto yang terhapus karena masuk ke dalam kriteria buruk di atas, saya langsung melakukan pengambilan foto ulang untuk menggantikannya. Hal ini sangat krusial dilakukan demi menjaga jumlah data tetap genap 24 gambar per kelas, serta memastikan bahwa sistem kecerdasan buatan nantinya hanya belajar dari sekumpulan data yang bersih, jelas, utuh, dan memiliki variasi yang kaya.

2. Kompresi Ukuran dan Standarisasi Format

Setelah mendapatkan 192 gambar uang yang bersih dan berkualitas baik, langkah selanjutnya adalah menyesuaikan ukuran dan format file tersebut. Tahapan ini saya lakukan secara terencana dengan rincian sebagai berikut:

a. Alasan Melakukan Kompresi

Foto asli yang dihasilkan dari kamera *smartphone* umumnya memiliki resolusi yang sangat tinggi dan ukuran file yang sangat besar (bisa mencapai beberapa Megabyte per foto). Jika saya langsung memasukkan ratusan foto berukuran besar ini untuk dipelajari oleh komputer, sistem akan bekerja terlalu berat, prosesnya menjadi sangat lambat, atau bahkan bisa mengalami *error* karena kehabisan memori. Oleh karena itu, saya perlu melakukan kompresi untuk memperingan beban kerja sistem.

b. Otomatisasi Menggunakan Python

Mengingat jumlah data yang diproses cukup banyak (192 foto), saya tidak melakukan pemotongan atau pengecilan gambar satu per satu secara manual. Saya menggunakan bahasa pemrograman Python untuk memproses seluruh gambar tersebut secara massal dan otomatis di dalam satu folder.

c. Standarisasi Ukuran (224x224 Pixel)

Seluruh gambar saya ubah ukurannya (*resize*) secara seragam menjadi bentuk persegi dengan dimensi 224x224 pixel. Ukuran ini sengaja saya pilih karena merupakan standar baku yang paling ideal dan umum digunakan saat kita melatih sistem kecerdasan buatan (AI) untuk mengenali sebuah gambar.

d. Standarisasi Format File (.JPG)

Kamera *smartphone* terkadang menghasilkan format foto yang berbeda-beda, seperti .PNG atau .JPEG. Untuk merapikan data, saya menyeragamkan seluruh file menjadi format .JPG. Saat proses penyimpanan kembali, saya mengatur

tingkat kualitas gambar di angka 85% (*quality=85*). Langkah ini sangat efektif karena ukuran file fotonya menyusut drastis menjadi sangat ringan, namun detail tulisan dan gambar pahlawan pada uang kertas tersebut tetap terlihat sangat jelas tanpa merusak informasi penting di dalamnya.

e. Sistem Pemantauan Otomatis

Untuk memastikan tidak ada file yang tertinggal atau gagal diproses, saya menambahkan sistem penghitung otomatis dan pendeteksi error pada program. Jika proses berhasil, sistem akan menampilkan centang hijau beserta nama filenya.

Berikut adalah implementasi kode Python yang saya gunakan untuk melakukan proses kompresi dan standarisasi secara massal tersebut:

```
from google.colab import drive
import os
from PIL import Image

# 1. Hubungkan Google Colab ke Google Drive
drive.mount('/content/drive')

# 2. Atur Lokasi Folder (KHUSUS KELAS 1000 TERANG)
# Asumsi: folder DATASET_DUIT ada di halaman depan
Google Drive kamu
folder_asal =
'/content/drive/MyDrive/DATASET_DUIT/5000_kertas/5000_
terang1'

# Kita buat folder penampungan baru bernama
DATASET_SIAP agar file asli aman
folder_tujuan =
'/content/drive/MyDrive/DATASET_DUIT/5000_kertas/5000_
terang'

# Perintah ini akan otomatis membuat folder tujuan
di Google Drive jika belum ada
if not os.path.exists(folder_tujuan):
    os.makedirs(folder_tujuan)

# 3. Proses Resize ke 224x224 dan Kompresi (Sesuai
instruksi dosen)
ukuran_target = (224, 224)
foto_berhasil = 0

print(f"Mulai memproses foto di
folder:\n{folder_asal}...\n")

# Cek semua file di dalam folder asal
try:
```

```
        for nama_file in os.listdir(folder_asal):
            # Hanya memproses file gambar
            if nama_file.lower().endswith(('.png',
'.jpg', '.jpeg')):
                path_asal = os.path.join(folder_asal,
nama_file)

                try:
                    # Buka gambar dan ubah ke format
standar RGB (wajib untuk standarisasi format)
                    img =
Image.open(path_asal).convert('RGB')

                    # Resize gambar jadi 224x224
                    img_resized =
img.resize(ukuran_target, Image.Resampling.LANCZOS)

                    # Pastikan format akhirnya seragam
menjadi .jpg (sesuai instruksi dosen)
                    nama_file_baru =
os.path.splitext(nama_file)[0] + '.jpg'
                    path_tujuan_baru =
os.path.join(folder_tujuan, nama_file_baru)

                    # Simpan dengan kualitas 85 (dari
10MB bisa jadi sekitar 30-50 KB saja!)
                    img_resized.save(path_tujuan_baru,
format='JPEG', quality=85)
                    foto_berhasil += 1

                    print(f" Berhasil kompres:
{nama_file_baru}")

                except Exception as e:
                    print(f" Gagal memproses file
{nama_file}: {e}")

        print(f"\n MANTAP! Total {foto_berhasil} foto
'5000 Terang' sukses dikompres & di-resize ke
224x224.")

    except FileNotFoundError:
        print(f" ERROR: Folder {folder_asal} tidak
ditemukan.")
        print("Pastikan folder DATASET_DUIT benar-benar
ada di halaman depan (root) Google Drive kamu, bukan
di dalam folder lain.")
```

3. Augmentasi Data untuk Modifikasi Kecerahan

Setelah ukuran file gambar menjadi jauh lebih ringan dan seragam, langkah selanjutnya yang saya lakukan adalah memperkaya variasi data. Tahapan ini biasa disebut dengan augmentasi data. Proses ini saya lakukan secara terstruktur dengan rincian penjelasan sebagai berikut:

a. Tujuan Melakukan Augmentasi

   Saya menerapkan tahapan ini untuk mensimulasikan kondisi transaksi yang sesungguhnya di lapangan. Karena pada kenyataannya, masyarakat sering menggunakan uang tunai di tempat yang minim pencahayaan, seperti di pasar tradisional pada saat subuh atau di warung tenda pada malam hari. Jika sistem komputer hanya diajarkan untuk mengenali uang dari foto yang sangat terang, maka sistem tersebut berpotensi gagal dan kebingungan saat harus mendeteksi uang di tempat yang redup.

b. Manajemen Penyimpanan yang Aman

   Sebelum program mengubah gambar, saya mengatur agar sistem membaca gambar dari folder sumber (misalnya folder 1000_terang), lalu secara otomatis membuat folder baru (folder 1000_gelap) untuk menyimpan hasilnya. Pemisahan folder ini saya lakukan agar file yang asli tidak tertimpa dan data tetap tertata dengan rapi di Google Drive.

c. Pemilihan Teknik Augmentasi

   Dari berbagai macam teknik augmentasi (seperti memutar, memotong, atau membalik gambar), saya memutuskan untuk hanya menggunakan teknik modifikasi kecerahan (*brightness*). Oleh karena itu, saya fokus untuk melatih komputer agar bisa melihat dan kebal terhadap perubahan cahaya, bukan perubahan posisi.

d. Penggunaan Faktor Kecerahan 0.2

   Melalui kode pemrograman, saya menurunkan tingkat cahaya dari seluruh gambar asli menggunakan faktor 0.2. Ini berarti saya hanya menyisakan sekitar 20% pencahayaan dari gambar aslinya. Angka ini saya pilih setelah melakukan uji coba, karena efek gelap yang dihasilkan terasa sangat pas; gambar terlihat seperti difoto di ruangan gelap, namun detail tulisan nominal dan gambar pahlawan pada uang kertas tersebut masih dapat dikenali dengan baik.

e. Efisiensi Penggandaan Data

Proses augmentasi ini berjalan secara otomatis. Program membaca 192 gambar terang yang sudah saya siapkan sebelumnya, lalu secara mandiri menciptakan kembaran dari gambar-gambar tersebut dalam versi yang gelap. Dengan memanfaatkan metode ini, saya dapat menghemat banyak waktu karena tidak perlu memotret ulang ratusan lembar uang satu per satu di ruangan yang gelap.

f. Hasil Akhir yang Seimbang (*Balance*)

Dataset yang awalnya berjumlah 192 gambar kini berhasil digandakan menjadi 384 gambar. Kondisi data yang dihasilkan sangat seimbang, di mana setiap kelas pecahan uang kertas kini memiliki porsi perwakilan yang sama persis dan adil, yaitu 24 gambar versi asli (terang) dan 24 gambar hasil simulasi (gelap).

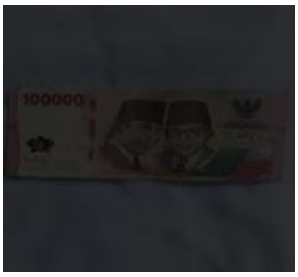
Berikut adalah implementasi kode Python yang saya gunakan untuk menerapkan efek modifikasi kecerahan tersebut:

```
from PIL import Image, ImageEnhance
import os

# 1. Pastikan Google Drive masih terhubung (jika
belum, jalankan drive.mount lagi)
try:
    if not os.path.exists('/content/drive'):
        from google.colab import drive
        drive.mount('/content/drive')
except:
    pass

# 2. Atur Lokasi Folder (KHUSUS KELAS 1000)
#   Kita   ambil   gambar   yang   sudah   di-resize
(DATASET_SIAP)
folder_sumber                                       =
'/content/drive/MyDrive/DATASET_DUIT/1000_kertas/1
000_terang'

# Kita buat folder tujuan baru di Drive (sejajar
dengan folder terang)
folder_gelap                                        =
'/content/drive/MyDrive/DATASET_DUIT/1000_kertas/1
000_gelap'

if not os.path.exists(folder_gelap):
    os.makedirs(folder_gelap)
```

```
# 3. Kunci Rahasia: Faktor Kegelapan
# Nilai 1.0 artinya gambar asli.
# Nilai di bawah 1.0 artinya semakin gelap.
# Nilai 0.0 artinya hitam total.
#  Untuk  efek  "ruangan  gelap  tapi  objek  masih
terlihat", nilai 0.2 - 0.3 adalah yang paling pas.
faktor_kegelapan = 0.2

foto_gelap_berhasil = 0

print(f"Mulai        menyulap        foto        di
folder:\n{folder_sumber}\nmenjadi gelap...\n")

# Cek semua file di folder sumber
try:
    for nama_file in os.listdir(folder_sumber):
        if      nama_file.lower().endswith(('.jpg',
'.jpeg', '.png')):
            path_asal = os.path.join(folder_sumber,
nama_file)

            try:
                # Buka gambar asli
                img = Image.open(path_asal)

                # Masuk ke mode augmentasi kecerahan
                enhancer                            =
ImageEnhance.Brightness(img)

                # Terapkan faktor kegelapan
                img_gelap                           =
enhancer.enhance(faktor_kegelapan)

                # Simpan dengan nama baru, contoh:
'gelap_foto1.jpg'
                nama_file_baru    =    'gelap_'    +
os.path.splitext(nama_file)[0] + '.jpg'
                path_tujuan_baru                    =
os.path.join(folder_gelap, nama_file_baru)

                # Simpan sebagai JPG
                img_gelap.save(path_tujuan_baru,
format='JPEG', quality=90)

                foto_gelap_berhasil += 1
                print(f"   Berhasil   menggelapkan:
{nama_file_baru}")

            except Exception as e:
                print(f"     Gagal     menggelapkan
{nama_file}: {e}")
```

```
        print(f"\n SUKSES BESAR! Total {foto_gelap_berhasil} foto '1000_gelap' baru berhasil dibuat.")
        print(f"Cek di Drive kamu, folder: DATASET_DUIT/1000_kertas/1000_gelap")

    except FileNotFoundError:
        print(f" ERROR: Folder sumber {folder_sumber} tidak ditemukan.")
    print("Pastikan kamu sudah menjalankan kode kompresi kelas 1000 sebelumnya.")
```

Dan berikut adalah contoh hasil augmentasi yang sudah saya lakukan:



4. Aturan Pelabelan (Labeling)

Setelah total 384 gambar uang kertas berhasil terkumpul (gabungan dari versi terang dan gelap), langkah penting selanjutnya adalah memberikan identitas yang jelas pada setiap gambar tersebut. Tahap pelabelan ini saya

lakukan sepenuhnya secara manual dan sangat teliti, dengan menerapkan dua pendekatan utama sebagai berikut:

a. Pengelompokan ke dalam Folder (*Directory-Based*)

   Pertama, saya memisahkan gambar-gambar tersebut ke dalam delapan folder utama yang namanya disesuaikan dengan nominal uangnya (misalnya folder 1000_kertas, 2000_kertas, dan seterusnya) yang didalamnya terdapat 2 folder untuk memisahkan terang dann gelap hasil augmentasi yang saya lakukan sebelumnya. Hal ini saya lakukan agar sistem komputer nantinya bisa langsung mengenali bahwa seluruh gambar di dalam folder tertentu adalah anggota dari kelas pecahan uang yang sama.

b. Penamaan File yang Deskriptif (*Descriptive File Naming*)

   Kedua, saya membuat aturan penamaan khusus yang konsisten sama untuk semua dataset saya, di mana nama file tersebut sekaligus menjadi rekam jejak yang mendeskripsikan ciri-ciri foto di dalamnya.

c. Format Kode Penamaan

   Agar seragam, saya menyusun format nama file dengan susunan baku yiatu [Nominal][Cahaya]_[Latar]_[Sisi]_[Angle].jpg. Berikut adalah rincian dari setiap komponen tersebut:

   - Nominal: Menunjukkan pecahan uang (1000, 2000, 5000, dst).
   - Cahaya: Menunjukkan kondisi pencahayaan, yaitu Trg (Terang) atau Glp (Gelap).
   - Latar: Menunjukkan jenis alas saat difoto, yaitu Pls (Polos) atau Mtf (Bermotif).
   - Sisi: Menunjukkan sisi uang yang difoto, yaitu Phl (Sisi depan bergambar Pahlawan) atau Bdy (Sisi belakang bergambar Kebudayaan/Pemandangan).
   - Angle: Menunjukkan sudut pengambilan gambar, seperti Dpn (Depan), Blk (Belakang), Kr (Kiri), Kn (Kanan), Dkt (Jarak Dekat), atau Jh (Jarak Jauh).

d. Contoh Pembacaan Label

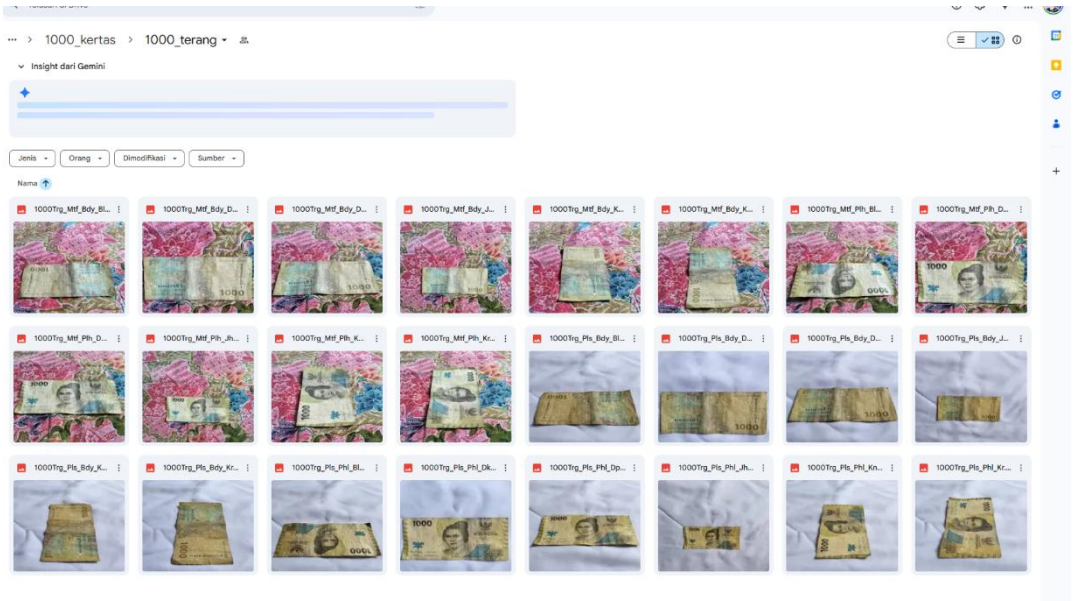
   Sebagai contoh, jika terdapat sebuah file bernama 1000Trg_Mtf_Phl_Dpn.jpg, maka siapa pun yang melihat nama file tersebut dapat langsung memahami

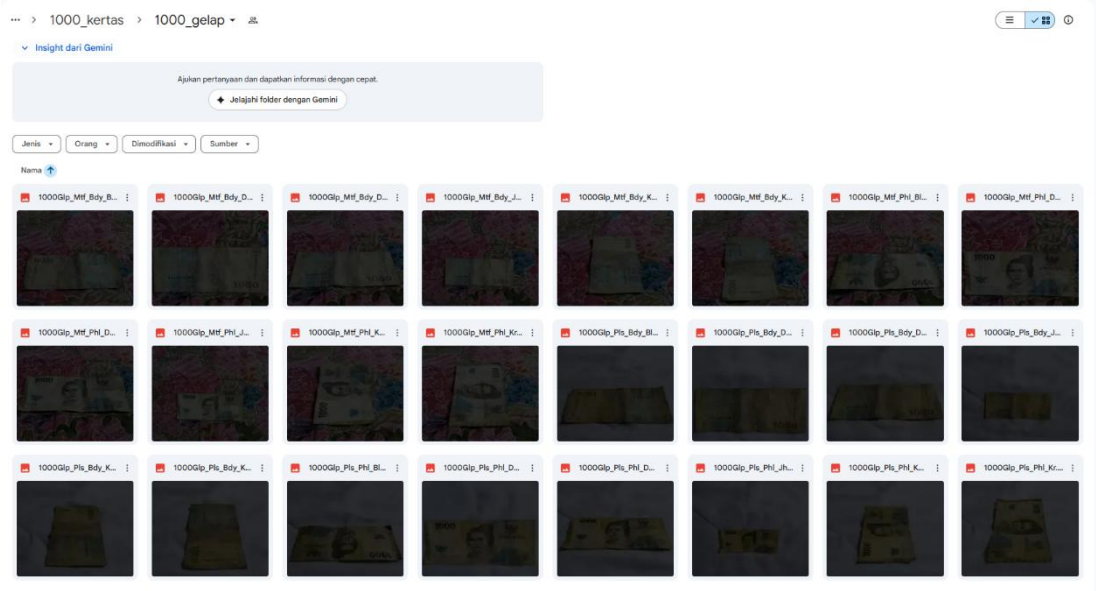
artinya secara spesifik. File tersebut berisi foto uang pecahan 1000, difoto dalam kondisi cahaya Terang, diletakkan di atas Latar Motif, menampilkan Sisi Pahlawan, dan difoto tepat dari sudut Depan.

e. Pelabelan dan Pengecekan Manual

Meskipun memakan waktu, saya sengaja melakukan penamaan ratusan file ini satu per satu secara manual. Tujuannya adalah untuk memastikan tingkat keakuratan mencapai 100%. Saya ingin menjamin tidak ada satu pun gambar yang salah masuk folder atau salah nama, karena kesalahan pelabelan sekecil apa pun di awal akan membuat sistem kecerdasan buatan belajar dari informasi yang keliru.

Berikut adalah gambaran aturan pelabelan dataset:





5. Standarisasi Channel Warna dan Normalisasi Nilai Pixel

Setelah seluruh proses pelabelan selesai, Langkah selanjutnya adalah mengubah gambar fisik menjadi data numerik (angka) agar bisa diproses oleh memori komputer. Proses ini saya lakukan dengan rincian sebagai berikut:

a. Otomatisasi Pembacaan Data

Saya merancang kode Python untuk masuk ke setiap folder nominal uang (dari 1.000 hingga 100.000) dan menyisir folder kondisi terang maupun gelap secara otomatis. Hal ini saya lakukan agar seluruh 384 gambar yang sudah saya siapkan dapat dimuat ke dalam sistem tanpa ada satu pun yang terlewat.

b. Koreksi Urutan Warna (BGR ke RGB)

Secara teknis, pustaka (*library*) OpenCV yang saya gunakan memiliki kebiasaan membaca urutan warna gambar secara terbalik, yaitu BGR (Biru-Hijau-Merah). Jika saya biarkan, maka warna uang kertas di mata komputer akan terlihat aneh dan tidak alami. Oleh karena itu, saya melakukan standarisasi dengan mengonversi urutan warnanya menjadi RGB (Merah-Hijau-Biru) agar sesuai dengan warna asli uang di dunia nyata.

c. Pengecekan Ulang Dimensi Gambar

Meskipun sebelumnya saya sudah melakukan *resize*, di tahap ini saya kembali menyisipkan fungsi pengecekan ukuran untuk memastikan seluruh gambar

yang masuk ke memori benar-benar memiliki dimensi 224x224 pixel karena agar tidak ada satu pun gambar dengan ukuran berbeda beda bisa masuk ke tahap pelatihan, karena perbedaan ukuran dapat menyebabkan sistem mengalami kesalahan (*error*).

d. Normalisasi Skala Piksel (0.0 - 1.0)

Komputer biasanya membaca intensitas warna dalam angka 0 sampai 255. Namun, dalam dunia kecerdasan buatan, angka yang terlalu besar cenderung membuat proses perhitungan menjadi sangat berat dan lama. Saya menerapkan metode Normalisasi, di mana nilai asli (0-255) tersebut saya bagi dengan angka 255.0. Hasilnya, angka-angka tersebut menyusut menjadi rentang 0.0 hingga 1.0. Dengan angka yang lebih kecil ini, sistem dapat belajar dengan jauh lebih cepat, ringan, dan akurat.

e. Verifikasi Hasil Akhir

saya juga menambahkan perintah untuk mencetak (*print*) rentang nilai piksel sebelum dan sesudah normalisasi. Hal ini saya lakukan sebagai bukti otentik bahwa proses transformasi data telah berhasil dilakukan dan dataset siap untuk masuk ke tahap pembagian (*splitting*).

Berikut adalah implementasi kode Python yang saya kembangkan untuk memuat data dan melakukan normalisasi tersebut:

```
import os
import cv2
import numpy as np

# 1. Menyiapkan lokasi dataset dan daftar kelas
path_dataset =
'/content/drive/MyDrive/DATASET_DUIT'
data_gambar = []
label_kelas = []
daftar_kelas = ['1000', '2000', '5000',
'10000', '20000', '50000', '75000', '100000']
daftar_kondisi = ['terang', 'gelap']
```

```
# 2. Looping otomatis untuk membaca semua
gambar di setiap sub-folder
for kelas in daftar_kelas:
    for kondisi in daftar_kondisi:
        nama_folder_anak = f"{kelas}_{kondisi}"
        folder_target =
os.path.join(path_dataset, f"{kelas}_kertas",
nama_folder_anak)

        if not os.path.exists(folder_target):
            continue

        for nama_file in
os.listdir(folder_target):
            if
nama_file.lower().endswith(('.jpg', '.jpeg',
'.png')):
                path_file =
os.path.join(folder_target, nama_file)
                img = cv2.imread(path_file)

                if img is not None:
                    # Menstandarisasi urutan
warna ke RGB
                    img = cv2.cvtColor(img,
cv2.COLOR_BGR2RGB)
                    # Memastikan kembali ukuran
tepat 224x224
                    img = cv2.resize(img, (224,
224))

                    data_gambar.append(img)
```

```
                        label_kelas.append(kelas)

# 3. Proses Normalisasi Pixel (Mengubah skala
0-255 menjadi 0-1)
X = np.array(data_gambar, dtype='float32')
y = np.array(label_kelas)

# Melakukan pembagian untuk normalisasi
X_normalisasi = X / 255.0

# Menampilkan bukti rentang nilai pixel
print(f" Rentang nilai pixel sesudah
normalisasi : {np.min(X_normalisasi)} sampai
{np.max(X_normalisasi)}"
```

6. Pembagian Dataset (Train-Test Split)

   Tahap akhir dari seluruh alur *preprocessing* ini adalah membagi dataset ke dalam dua kelompok yang fungsinya berbeda. Adapun rincian sebagai berikut:

   a. Tujuan Pembagian Data

      Saya harus membagi menjadi dua kelompok yaitu satu kelompok untuk latih dan satu kelompok lagi sebagai data uji. Hal ini agar saya bisa mengetahui sistem yang dibuat benar-benar sudah pintar bukan cuman sekadar menghafal gambar.

   b. Penentuan Rasio (80% : 20%)

      Saya memutuskan untuk menggunakan rasio yang paling umum dan teruji dalam dunia *Machine Learning*, yaitu 80% untuk Data Latih (*Training Data*) dan 20% untuk Data Uji (*Testing Data*). Dengan pembagian ini, komputer memiliki cukup banyak materi (307 gambar) untuk dipelajari, dan tetap memiliki tantangan ujian (77 gambar) yang memadai untuk membuktikan akurasinya.

c. Penerapan Teknik *Stratify*

Saya menggunakan parameter *stratify* untuk menjamin keadilan distribusi datanya. Dengan teknik ini, sistem secara otomatis memastikan bahwa di dalam kelompok sata uji, setiap pecahan uang dari Rp1.000 hingga Rp100.000 diwakili secara adil.

d. Menjaga Konsistensi dengan *Random State*

Saya menambahkan random_state=42 di dalam kode saya. Tujuannya agar hasil pembagian data tetap konsisten dan tidak berubah-ubah setiap kali program dijalankan ulang.

e. Verifikasi Distribusi Akhir

Setelah pembagian selesai, saya menambahkan kode untuk menghitung secara otomatis berapa banyak gambar dari tiap pecahan uang yang masuk ke kelompok ujian. Berdasarkan hasil pengecekan sistem, saya berhasil mendapatkan distribusi yang sangat seimbang, di mana setiap nominal uang kini diwakili oleh 9 hingga 10 gambar uji secara adil.

Berikut adalah implementasi kode Python yang saya gunakan untuk melakukan pembagian dataset tersebut:

```
from sklearn.model_selection import
train_test_split
import collections

# Membagi data secara otomatis: 80% untuk
Training, 20% untuk Testing
X_train, X_test, y_train, y_test =
train_test_split(
    X_normalisasi,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y # Menjamin pembagian per kelas
tetap seimbang (balance)
)

print(" PROSES SPLIT DATA SELESAI!")
print(f"Data untuk Belajar (Training) :
{len(X_train)} gambar (80%)")
print(f"Data untuk Ujian (Testing)   :
{len(X_test)} gambar (20%)")
```

```
    print("\n Detail Jumlah Data Ujian per Kelas
    (Testing):")
    # Menghitung dan menampilkan distribusi data
    testing per nominal
    hitung_test = collections.Counter(y_test)
    for kelas, jumlah in
    sorted(hitung_test.items()):
   print(f"Kelas Rp {kelas}: {jumlah} gambar")
```

4. KESIMPULAN

Secara keseluruhan, tahapan *preprocessing* dan augmentasi dataset klasifikasi 8 nominal uang kertas Rupiah telah selesai diimplementasikan. Melalui proses penyortiran data, penyesuaian ukuran menjadi 224x224 pixel, dan penambahan variasi cahaya gelap, dataset awal sebanyak 192 gambar telah berhasil digandakan menjadi 384 gambar. Dataset akhir ini memiliki distribusi kelas yang seimbang dan telah melewati proses normalisasi piksel. Selanjutnya, data dibagi menggunakan rasio 80:20, menghasilkan 307 gambar sebagai Data Latih dan 77 gambar sebagai Data Uji. Hasil pemrosesan ini diharapkan sudah cukup memadai untuk digunakan sebagai data dasar (*baseline*) pada tahap pelatihan model klasifikasi (*Machine Learning*) selanjutnya.

Link Presentasi:
https://www.canva.com/design/DAHDEf0eN0Y/y2bBWQcC6Wu78nyxWwCTtg/edit?utm_content=DAHDEf0eN0Y&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Link Github: *https://github.com/dinzzti/Machine_Learning_Money.git*

REFERENSI

[1] D. Hasiana, “BI: Perputaran Uang Sepanjang 2024 Rp1.204 Triliun, Melonjak 9,3%.” [Online]. Available: https://www.bloombergtechnoz.com/businessweek

[2] F. K. Sari, M. A. Ghofur, and F. K. Sari, “Analysis of the effect of non-cash payment systems on Indonesia ’ s economic growth for the period 2013-2023,” *Digit. Bus. Tren Bisnis Masa Depan*, vol. 16, no. 1, pp. 9–15, 2025, doi: https://doi.org/10.59651/digital.v16i1.249.

[3] S. M. Bahrani, T. Shahani, and E. Collegemumbai, “Deep Learning Approach for Indian Currency Clasification,” vol. 5, no. 6, pp. 335–340, 2020, doi: https://doi.org/10.33564/IJEAST.2020.v05i06.049.

[4] C. Mankar, A. Agrawal, V. Tiwari, and N. Shrinath, “Currency Detector Android Application for Visually Impaired People,” *Int. J. Adv. Res. Sci. Commun. Technol.*, pp. 489–496, 2023, doi: 10.48175/IJARSCT-9606.